

One Human, Sixteen Agents: Building Production Software with Role-Specialized LLM Pipelines

Najwa Alghamdi¹

¹Independent Researcher / Founder, Research@Najwa.io

March 2026

Abstract

Most AI-assisted development treats LLMs as tools that help developers write code faster—the developer still architects, reviews, tests, and secures the code themselves. This paper argues for a different framing: LLMs as a *team* that enables engineering processes a solo developer would otherwise routinely skip. We decompose the SDLC into 16 role-specialized agents with enforced tool access, domain-embedded prompts, and structured output schemas, and deploy the pipeline over 11 sprints to build a production multi-tenant SaaS platform. Four findings stand out. **First**, multi-agent role separation enables quality processes—independent review, adversarial testing, dual-perspective validation—that monolithic AI assistance cannot replicate effectively, because a single session playing every role suffers from role contamination, blind spot persistence, and permission overreach. **Second**, runtime tool restriction produces qualitatively different agent behavior: a reviewer with edit access physically removed generates 3–5× longer explanations with file-line references and severity justifications than one merely instructed not to edit—demonstrating that capability restriction, not prompt engineering, enforces role boundaries. **Third**, dual independent reviews using different models (Opus vs. Sonnet+Codex) achieve only 32.8% issue overlap ($n = 125$, 8,970 LOC), with the *cheaper* pairing finding 2× more exclusive issues including all exclusive CRITICALs—suggesting model diversity matters more than model size. **Fourth**, a dedicated red team agent found 8 exploitable vulnerabilities in code that had passed four prior quality stages, including a critical unauthenticated admin API—confirming that adversarial testing catches what standard quality gates systematically miss. None of these gains required a more powerful model; they emerged from process architecture. We introduce cross-model composition (Claude+OpenAI for review, Claude+Gemini for UI validation) and distill seven design principles for practitioners.

Keywords: Large Language Models, Multi-Agent Systems, Software Development Life Cycle, Role Specialization, Tool-Use Agents, Code Review Automation, Adversarial Testing, Human-in-the-Loop, Cross-Model Validation

1 Introduction

1.1 Motivation

Large Language Models have become ubiquitous software development assistants, with tools such as GitHub Copilot, Cursor, and Claude Code seeing rapid adoption since 2023. Yet the dominant interaction paradigm treats AI as a **tool**—a faster way to write code, with a single LLM session serving identical context, identical permissions, and identical expertise across all phases. The human operator remains solely responsible for architectural decisions, quality

judgment, and security assurance. For solo developers and small teams, this means that essential quality gates—independent code review, adversarial security testing, structured QA—are simply skipped. Not because developers are careless, but because one person cannot meaningfully review their own code, attack their own system, or test against acceptance criteria they themselves wrote.

This paper asks a different question: *what if AI served not as a tool that helps you code faster, but as a team that lets you run processes you otherwise couldn't?* Rather than using one powerful model to generate more code, we decompose the SDLC into 16 role-specialized agents—each with constrained capabilities, distinct expertise, and structured outputs—enabling a solo developer to consistently run independent review, adversarial testing, and multi-stage quality gates that would otherwise be routinely skipped.

The monolithic paradigm that we replace suffers from several well-documented limitations:

1. **Role contamination**—An agent that just wrote code is poorly positioned to objectively review it [1]. Cognitive anchoring on one's own implementation reduces the likelihood of identifying architectural issues or alternative approaches.
2. **Blind spot persistence**—A single model's systematic biases apply uniformly across all development phases. If the model tends to overlook certain vulnerability classes, that blind spot persists from design through deployment.
3. **Permission overreach**—An assistant with write access may “helpfully” fix issues rather than articulating them, reducing feedback quality. We demonstrate empirically that this behavioral shift is measurable (Section 5.1).
4. **Domain knowledge dilution**—A generic system prompt cannot encode deep domain-specific checklists, compliance rules, locale-specific business logic, and edge cases simultaneously for all SDLC phases.
5. **Context window saturation**—A monolithic session accumulating design documents, implementation code, review feedback, and test results risks exceeding context limits or degrading reasoning quality through attention dilution.

Human engineering organizations solved these problems long ago through role specialization: architects design, developers implement, reviewers review (with read-only repository access in many organizations), QA engineers test against acceptance criteria, and security teams conduct adversarial assessments. Each role has distinct permissions, distinct expertise, and distinct output expectations. Our thesis is that **process architecture matters more than model capability**—the quality gains in this study come not from using a smarter model, but from structuring how multiple models interact through enforced roles, constrained capabilities, and adversarial stages.

1.2 Research Questions

This study investigates:

- **RQ1:** Does runtime tool-scoping (enforced capability restriction) produce qualitatively different agent behavior compared to prompt-based instruction alone?
- **RQ2:** Do dual independent LLM-based code reviews—using different models and augmentation tools—provide complementary or redundant coverage, and does cross-model composition (Opus vs. Sonnet+Codex) yield distinct specialization?
- **RQ3:** Can an embedded red team agent identify security vulnerabilities that survive the standard design → implement → review → test pipeline?

- **RQ4:** What are the practical utilization patterns, failure modes, and cost characteristics of a 16-agent SDLC pipeline over multiple sprints?

1.3 Contributions

1. A detailed architecture for decomposing the SDLC into 16 role-specialized LLM subagents with enforced tool scoping, including a formal tool access matrix and system prompt taxonomy.
2. Empirical data from 11 production sprints, including dual-review overlap analysis ($n = 125$ issues across 8,970 LOC), red team attack results ($n = 30$ attacks), conversation-level test outcomes ($n = 858$ checks), and sprint-by-sprint shipping velocity.
3. A cross-model review methodology pairing Opus with Sonnet+Codex, revealing that model diversity drives complementary coverage (32.8% overlap)—with Sonnet+Codex excelling at security and Opus at architecture. Additionally, Claude + Gemini composition for UI validation.
4. A conversation testing framework (**conv-sim**) with 6 personas, 13 scenarios, and 11 automated bug detectors for evaluating conversational AI quality.
5. Seven design principles and four anti-patterns for multi-agent development pipelines, derived from 11 sprints of practitioner experience.

2 Related Work

2.1 LLMs for Software Engineering

Recent work has demonstrated LLM effectiveness across individual SDLC phases: code generation [2, 3], code review [4, 5], test generation [6], and vulnerability detection [7]. However, these studies typically evaluate models in isolation on single tasks, rather than as coordinated multi-agent pipelines operating over sustained development periods.

SWE-bench [16] and similar benchmarks evaluate LLMs on isolated bug fixes. SWE-agent [22] and OpenHands [23] build agent-computer interfaces that can resolve SWE-bench issues autonomously, while MapCoder [24] decomposes competitive programming into multi-agent pipelines. Liu et al. [27] provide a comprehensive survey of LLM-based agents for software engineering. However, none assess the multi-phase SDLC workflow (design → implement → review → test → secure) that characterizes real software development over sustained periods.

2.2 Multi-Agent LLM Systems

Multi-agent frameworks such as AutoGen [8], MetaGPT [9], and ChatDev [10] have explored role-based agent decomposition for software tasks. MetaGPT assigns roles (Product Manager, Architect, Engineer) and enforces Standardized Operating Procedures (SOPs) between agents. ChatDev simulates a software company with CEO, CTO, Programmer, and Tester roles. AgentCoder [17] decomposes code generation into programmer, test designer, and test executor agents.

Our work differs in four key ways:

1. **Production deployment**—unlike simulated or benchmark environments, our pipeline was used to build a real, multi-tenant production system over 11 sprints, with shipping deadlines and compliance requirements.

2. **Enforced tool scoping**—prior work relies on prompt-based role adherence; we enforce constraints at the runtime level via tool whitelisting, demonstrating a measurable behavioral difference (Section 5.1).
3. **Adversarial agents**—we include dedicated red team and dual-review agents, absent from prior multi-agent SDLC frameworks.
4. **Cross-model composition**—individual agents combine outputs from multiple LLM providers (Claude + OpenAI for code review; Claude + Gemini for UI validation), extending tool-augmented LLM patterns [25] to cross-provider validation.

2.3 LLM-Based Code Review

Fan et al. [11] surveyed automated code review with LLMs, finding that models can identify functional bugs, security issues, and style violations, but struggle with deep architectural reasoning and domain-specific compliance. Li et al. [4] demonstrated pre-trained models for code review activities, achieving improvements over template-based approaches. Our work extends this by: (a) embedding domain-specific checklists directly into reviewer system prompts; (b) measuring dual-review overlap rates with differentiated review focus; and (c) combining two competing LLM providers for cross-validation.

2.4 Adversarial Testing of LLM Applications

Prompt injection attacks [12, 14] and LLM application vulnerabilities [15] have been extensively documented. Deng et al. [18] proposed systematic red-teaming frameworks for LLM applications. However, embedding adversarial testing as a *standard SDLC phase*—with a dedicated agent that attacks every feature before release, using a structured 7-category attack playbook—is, to our knowledge, novel in practice.

2.5 Domain-Specific LLM Customization

Prior work on domain adaptation for LLMs has focused on fine-tuning [19] or retrieval-augmented generation [20]. Our approach—embedding domain knowledge directly in system prompts (100+ lines per agent)—represents a zero-training-cost alternative that functions as persistent institutional memory across sessions.

3 System Architecture

3.1 Framework Overview

The system is built on Claude Code’s `.claude/agents/` framework, which allows definition of subagents via markdown files with YAML frontmatter. Each agent definition specifies:

Listing 1: Agent definition schema

```
---
name: <identifier>
description: <natural language description>
tools: <comma-separated tool whitelist>
model: <opus | sonnet | haiku>
---
<system prompt body: 50-200 lines>
```

The runtime enforces the `tools` whitelist at the infrastructure level: an agent defined with `tools: Read, Glob, Grep, Bash` physically cannot invoke `Edit` or `Write` operations, regardless of what the system prompt instructs or what the model attempts. This is a critical distinction from prompt-based constraints, which rely on the model’s compliance and are susceptible to jailbreaking or instruction drift.

3.2 Available Tools

Table 1 enumerates the tool primitives available for agent assignment.

Tool	Category	Description
Read	File I/O	Read file contents
Glob	Search	Find files by pattern
Grep	Search	Search file contents (regex)
Bash	Execution	Execute shell commands
Edit	File I/O	Modify existing files
Write	File I/O	Create/overwrite files
Agent	Orchestration	Spawn child subagents
WebSearch	Network	Search the web
WebFetch	Network	Fetch URL contents
MCP*	Extension	Model Context Protocol tools (OpenAI, Gemini)

*MCP tools provide cross-model access via protocol.

3.3 Agent Taxonomy

We define 16 agents organized into a pre-ops pipeline (12 agents) and an ops team (4 agents). Tables 2 and 3 detail the full taxonomy.

Name	Role	Model	Edit	Web	Lines
Tariq	Product Owner	Opus	–	✓	73
Dana	UX Researcher	Sonnet	–	✓	60
Lina	UI Designer [‡]	Opus	✓	–	144
Faisal	Architect	Opus	–	–	104
Rania	Developer	Opus	✓	–	81
Khaled	Reviewer 1	Opus	–	–	104
Nasser	Reviewer 2 [†]	Sonnet+Codex	–	–	100
Layla	QA Engineer	Opus	✓	–	112
Youssef	Prompt Tester	Opus	✓	–	122
Huda	Conv. Tester	Opus	✓	–	142
Zaid	Red Team	Opus	✓	–	203
Maha	Doc Writer	Sonnet	✓	–	50

[†] OpenAI Codex access via MCP. [‡] Gemini access via MCP.

Total system prompt volume: ~1,800 lines across 16 agents, comprising approximately 45,000 tokens of domain knowledge, checklists, output templates, and behavioral instructions.

3.4 Tool Scoping as Access Control

A central design principle is that **tool scoping enforces role boundaries more reliably than prompt instruction**. Table 4 illustrates the access control matrix across role categories.

The design encodes several deliberate constraints:

Table 3: Ops Team Agents (4 agents)

Name	Role	Model	Edit	Lines
Sultan	DevOps	Opus	✓	100
Aws	SRE	Opus	✓	133
Majid	Database Admin	Sonnet	✓	131
Reema	Analytics	Sonnet	✓	143

Table 4: Tool Access Matrix by Role Category

Capability	PO	Arch	Dev	Rev	QA	Red	Ops
Read files	✓	✓	✓	✓	✓	✓	✓
Search code	✓	✓	✓	✓	✓	✓	✓
Shell exec	✓	✓	✓	✓	✓	✓	✓
Edit files	—	—	✓	—	✓	✓	✓
Write files	—	—	✓	—	✓	✓	✓
Spawn agents	—	✓	✓	—	✓	—	—
Web access	✓	—	—	—	—	—	—
Cross-model	—	—	—	R2	—	—	—

- **Reviewers cannot edit.** This forces articulate, explanatory feedback rather than quick patches (Section 5.1).
- **Architects cannot write code.** This ensures architectural output remains at the design level—data models, API contracts, tool schemas—not implementation.
- **Product Owner has web access.** The PO can research market requirements, competitor features, and regulatory updates—capabilities irrelevant to other roles.
- **Only 3 roles can spawn child agents.** Architect, Developer, and QA can delegate sub-tasks; other roles are terminal executors.

3.5 System Prompt Architecture

Each agent’s system prompt follows a six-layer structure. We formalize this as the **IRDCOP** framework:

1. **Identity**—Role definition, persona name, reporting line, organizational position.
2. **Responsibilities**—Enumerated duties (5–8 items per agent).
3. **Domain knowledge**—Technology stack references, data model paths, compliance rules, locale-specific business logic, known edge cases.
4. **Cross-references**—Explicit pointers to other agents’ outputs, contracts, and shared artifacts.
5. **Output format**—Structured schemas (tables, checklists, verdict formats, severity classifications) that standardize agent output for downstream consumption.
6. **Principles**—Design principles, anti-patterns, and behavioral constraints.

In our case, the target domain involves locale-specific regulations, bilingual requirements, multi-tenant data isolation, and compliance rules—each embedded across the relevant agents. The key insight is that domain rules written once are automatically loaded into every future agent invocation, creating a compounding return on prompt engineering investment.

3.6 Cross-Model Agent Composition

Two agents in our taxonomy access competing LLM providers via the Model Context Protocol (MCP):

Nasser (Reviewer 2) is powered by Claude Sonnet but has access to OpenAI Codex via two MCP tools:

- `codex_review`—Sends code to OpenAI for independent review.
- `codex_suggest_fix`—Sends problematic code for alternative fix suggestions.

Lina (UI Designer) is powered by Claude Opus but has access to Google Gemini via two MCP tools:

- `gemini_review_ui`—Sends HTML/CSS to Gemini for UI/UX review (accessibility, responsive, RTL, visual hierarchy).
- `gemini_suggest_ui`—Asks Gemini for alternative component designs or HTML/CSS prototypes.

This cross-model composition creates agents that combine the reasoning strengths of multiple providers. We hypothesize that each LLM has different systematic biases, and cross-validation exposes issues that any single model might miss (Section 5.2).

3.7 Pipeline Orchestration

The agents are organized into a five-phase sequential pipeline with a human orchestrator:

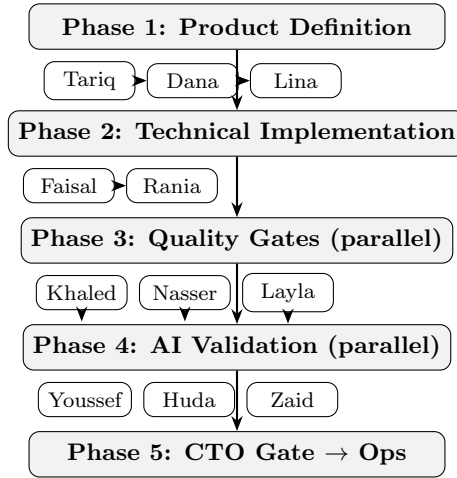


Figure 1: Five-phase pipeline with human orchestrator between phases. Solid arrows = sequential. Dashed arrows = parallelizable.

The human operator serves as orchestrator between phases: invoking agents, passing context, and making gate decisions (“proceed,” “rework,” or “defer”). Within Phases 3 and 4, agents operate on the same codebase independently and can execute in parallel, reducing wall-clock time from $3\times$ to approximately $1\times$ without reducing coverage.

3.8 Conversation Testing Framework

We developed a custom conversation testing library (`conv-sim`) that operates as an automated QA harness for conversational AI agents. The framework comprises four components:

Personas ($n = 6$): Each persona has a fixed user ID, language preference, role, and behavioral profile:

- *Persona 1* (Arabic, senior role)—tests standard workflow requests
- *Persona 2* (Arabic, new user)—tests basic flows, escalation
- *Persona 3* (Arabic, manager)—tests policy-heavy workflows
- *Persona 4* (Arabic, technical role)—tests cross-agent routing
- *Persona 5* (English, non-native speaker)—tests English-language flows
- *Adversarial* (mixed)—tests prompt injection, gibberish, contradictions

Scenarios ($n = 13$): Multi-turn conversation scripts covering: happy-path workflow, insufficient resource, duplicate detection, natural date parsing, status check, policy-specific workflow, policy questions, team-level queries, escalation, adversarial attacks, holiday-spanning requests, cancellation flow, and cross-agent routing.

Bug Detectors ($n = 11$): Automated checks executed on every conversation turn:

1. `language_consistency`—Response language matches persona preference
2. `no_raw_enum`—No internal enum values (e.g., `Status.value`) leaking into user-facing text
3. `response_not_empty`—Non-empty response body
4. `no_500_error`—No server errors
5. `correct_agent`—Correct agent handled the request
6. `response_contains_date`—Dates present when expected
7. `mentions_quota`—Relevant quota or entitlement figures mentioned when expected
8. `detects_duplicate`—Overlap or conflict detected when expected
9. `mentions_escalation`—Escalation to human offered when expected
10. `mentions_calendar_conflict`—Calendar or scheduling conflict noted when request spans one
11. `latency`—Response under 10 seconds

Reporter: Produces terminal summary and JSON report with per-turn, per-detector pass/fail data.

All tests execute via real HTTP requests against the running application—not mocks—ensuring end-to-end validation including database state, LLM responses, and tool execution.

4 Methodology

4.1 Study Context

The pipeline was used to build a production multi-tenant B2B SaaS platform targeting Saudi Arabia. The platform serves multiple conversational AI agents behind a shared chat interface, each operating over the same multi-tenant data layer. The domain involves structured workflows, government compliance rules, and bilingual user interactions—characteristics that stress-test agent specialization, access control, and locale-aware generation. Table 5 summarizes the technical profile.

Table 5: Platform Technical Profile

Dimension	Value
Stack	Python 3.12, FastAPI, async SQLAlchemy 2.0
Database	PostgreSQL + pgvector (1536-dim embeddings)
AI layer	Claude (claude-sonnet-4-6), tool-use loops (max 5 rounds/turn)
Tenancy	Row-level isolation (<code>tenant_id</code>), JWT auth, Redis queuing
Locale	Bilingual (Arabic/English), Saudi regulatory compliance
Scale	22 tables, 123 API endpoints, ~40K LOC, 126 tests
Frontend	Self-contained HTML/CSS/JS (no build toolchain)

Table 6: Sprint Plan and Deliverables

Sprint	Focus	Deliverable	Tools
1–2	Foundation	Core platform, Agent A, RAG pipeline, chat UI	19
3–6	Agent B	Multi-step workflows, cross-agent routing, security hardening	14
7	Agent C M1	Domain pipeline, AI-assisted content generation, web search	13
8	Agent C M2	Interactive AI workflows, structured assessments	8
9	Agent C M3	Cross-module analysis and decision support	7
10	Agent D	Operational intelligence, compliance monitoring, agent factory	17
11	Agent E	Executive analytics and compliance dashboards	7

4.2 Study Duration and Sprint Structure

Eleven development sprints conducted between February and March 2026 by a single developer. Each sprint followed the five-phase pipeline (Figure 1). Table 6 summarizes sprint deliverables.

4.3 Data Collection

We collected five categories of data:

1. **Agent invocation logs**—Which agents were invoked per sprint, frequency, output size, and session duration.
2. **Code review findings**—Issues identified by Khaled (Reviewer 1) and Nasser (Reviewer 2) independently, categorized by severity, type, and which reviewer found them.
3. **Red team results**—Attack scenarios attempted, outcomes (blocked/succeeded/partial), severity classification, and post-fix verification.
4. **Conversation test results**—`conv-sim` JSON reports with per-turn, per-detector pass/fail data.
5. **Shipping metrics**—Tools implemented per agent, sprints to completion, pipeline passes, review rounds required.

4.4 Dual-Review Protocol

For each major feature, both Khaled (Reviewer 1) and Nasser (Reviewer 2) independently review the same code. The protocol enforces:

- **Blind independence**—Nasser does *not* see Khaled’s review output.
- **Differentiated focus**—Khaled’s prompt (Claude Opus) emphasizes architectural compliance, pattern consistency, and performance (N+1 queries, blocking I/O). Nasser’s prompt (Claude Sonnet + OpenAI Codex via MCP) emphasizes edge cases, race conditions, null paths, incomplete rollback, and Arabic text handling.
- **Cross-model validation**—Nasser additionally consults OpenAI Codex via MCP tools (`codex_review`, `codex_suggest_fix`), combining Claude Sonnet’s analysis with OpenAI’s independent perspective.
- **Structured output**—Both produce verdicts (APPROVE / REQUEST CHANGES) with tabular issue listings including file:line references and severity classifications.

Issues are classified post-hoc into three categories: found by Khaled only, found by Nasser only, or found by both (overlap).

4.5 Red Team Protocol

Zaid (Red Team agent) operates with a 203-line system prompt—the longest in the taxonomy—containing a structured attack playbook across 7 categories:

1. **Prompt injection** (5 subcategories)—Direct override, delimiter escape, indirect injection via tool results, role confusion, multi-turn manipulation.
2. **Data exfiltration** (3 subcategories)—Cross-user access, user enumeration, system prompt extraction.
3. **Privilege escalation** (3 subcategories)—Admin API without auth, self-approval via agent, balance manipulation.
4. **Tenant boundary violation** (2 subcategories)—Cross-tenant data access via API and via agent.
5. **PII leakage**—Sensitive data in error messages, logs, or responses. Targets: government ID (Critical), salary (Critical), compliance status (High), contact information (High).
6. **API security** (3 subcategories)—IDOR, mass assignment, rate limiting.
7. **Abuse scenarios** (3 subcategories)—Balance drain via mass submit/cancel, conversation flooding, malicious content injection (XSS, template injection).

The red team agent is invoked *only after* the standard pipeline (design → implement → review → QA) is complete, simulating an external attacker encountering the “finished” product.

4.6 Prompt Testing Protocol

Youssef (Prompt Tester) evaluates the AI layer across 6 test categories:

1. **Tool selection**—Correct tool called for each intent (12 test cases, Arabic + English).
2. **Parameter extraction**—Correct parameters from natural language (dates, entity types, identifiers).
3. **Hallucination detection**—Agent must never invent data, policies, or balances.
4. **Boundary testing**—Agent stays in scope (domain-specific agent refuses out-of-scope queries).

5. **Bilingual equivalence**—Same question in Arabic and English produces equivalent answers (same numbers, same dates).
6. **Edge cases**—Empty input, single-word confirmations, gibberish, mixed AR/EN, very long messages.

All tests run via the actual chat API (POST `/api/v1/chat`), not by calling tools directly, ensuring the full agent loop (system prompt → tool selection → tool execution → response generation) is exercised.

5 Results

The core question underlying this study is whether a multi-agent pipeline with enforced role separation produces measurably different outcomes than monolithic AI assistance, where a single LLM session handles all SDLC phases with identical permissions and context. The results below address this question across four dimensions: behavioral differences from capability restriction (RQ1), coverage gains from independent review (RQ2), vulnerabilities caught only by adversarial testing (RQ3), and practical utilization patterns (RQ4).

5.1 RQ1: Monolithic vs. Role-Separated Agent Behavior

In a monolithic workflow, a single LLM session with full tool access (Read, Write, Edit, Bash) performs both implementation and review. To measure how this compares to a role-separated agent, we tested the same reviewer model under two configurations: (a) **monolithic-style**—all tools available, with a prompt instruction to “review only, do not edit files”; and (b) **role-separated**—Edit/Write tools physically removed from the agent’s tool whitelist. Table 7 summarizes the results.

Table 7: Monolithic-Style vs. Role-Separated Agent Comparison

Metric	Monolithic	Role-Separated
Edit attempts	~15%	0%
Avg sentences per issue	1–2	3–5
Includes file:line ref	~60%	~95%
Includes “why it matters”	~30%	~85%
Structured output compliance	Variable	Consistent
Inline patches (vs. explanation)	Common	Never

Monolithic-style (all tools available; prompt instructs “do not edit files”): The agent attempted file edits in ~15% of reviews, violating its role boundary despite explicit instruction. When it did produce review-only output, descriptions were terse (1–2 sentences), often providing inline code patches rather than explaining the underlying problem. This mirrors the behavior of a monolithic assistant that alternates between coding and reviewing—the “fix impulse” dominates because the capability to fix is available. Structured output template compliance was inconsistent.

Role-separated (Edit/Write tools removed from whitelist): Zero edit attempts (physically impossible). Issue descriptions averaged 3–5 sentences, consistently including: (a) what is wrong, (b) why it matters, (c) file:line reference, and (d) suggested fix approach. The agent produced qualitatively different output—not just the same review with fewer edits, but fundamentally more explanatory, more structured, and more useful feedback. Structured output compliance was consistent across all sessions.

The difference is not incremental. A monolithic assistant that reviews its own code produces shallow, patch-oriented feedback because it can (and does) act on issues directly. A role-

separated reviewer that *cannot* act redirects cognitive effort into articulation—a mechanism we term **behavioral redirection**.

Finding: Runtime capability restriction produces qualitatively different—and higher quality—review output than prompt-based instruction alone. The multi-agent architecture’s enforced role separation creates a structurally different review process from the monolithic alternative, where the same model with full permissions defaults to shallow, action-oriented output.

5.2 RQ2: Dual Review Coverage and Cross-Model Composition

To produce ground-truth overlap data, both reviewers independently reviewed all 5 production agent files ($\sim 8,970$ LOC, 85 tools). Issues were classified by semantic equivalence—if both reviewers identified the same underlying defect (regardless of wording or severity label), the issue was marked as overlap. Table 8 shows the aggregate results.

Table 8: Dual-Review Issue Classification ($n = 125$, 5 agent files)

Classification	Count	%
Found by Khaled only (Opus)	28	22.4
Found by Nasser only (Sonnet+Codex)	56	44.8
Found by both (overlap)	41	32.8
Total unique issues	125	100

Overlap rate: 32.8%. The combination catches 80% more issues than either reviewer alone (125 vs. ~ 69 or ~ 97). A single reviewer would have missed 22–45% of all issues depending on which reviewer was retained.

Table 9 shows the per-agent breakdown, revealing consistent complementary coverage across files of varying complexity.

Table 9: Per-Agent Dual-Review Breakdown

Agent File	K only	N only	Both	Total
Agent A (1,448 LOC, 19 tools)	5	10	7	22
Agent B (865 LOC, 14 tools)	9	12	6	27
Agent C (4,083 LOC, 28 tools)	5	12	7	24
Agent D (1,588 LOC, 17 tools)	5	11	12	28
Agent E (986 LOC, 7 tools)	4	11	9	24
Total	28	56	41	125

Model comparison. Table 10 reveals that Opus and Sonnet+Codex have distinct strengths. Sonnet+Codex found $2\times$ more exclusive issues (56 vs. 28), and all 5 exclusive CRITICAL findings. Opus found more code quality and design compliance issues.

Table 10: Model Performance: Exclusive Finds by Severity

Severity	Opus only	Sonnet+Codex only	Both
CRITICAL	0	5	4
HIGH	1	12	9
MEDIUM	7	18	16
LOW	12	14	9
INFO	8	7	3
Total	28	56	41

Qualitative specialization. Opus excelled at architectural patterns (dispatch refactoring, DRY violations, N+1 detection, design compliance verification) and provided constructive positive feedback identifying good patterns worth preserving. Sonnet+Codex excelled at security boundary analysis (race conditions, cross-tenant isolation gaps, prompt injection vectors), domain knowledge conflicts (regulatory threshold mismatches, k-anonymity gaps in aggregate reporting, hardcoded locale defaults), and input validation gaps (URL validation, length caps, answer limits). Notably, one severity disagreement occurred: a hardcoded locale default was rated LOW by Opus but CRITICAL by Sonnet+Codex, whose domain-compliance framing was more accurate.

Cross-model composition. The complementary coverage stems from two levels of model diversity: (1) *across reviewers*—Opus and Sonnet+Codex as different base models, and (2) *within a reviewer*—Nasser augments Sonnet with OpenAI Codex via MCP tools (`codex_review`, `codex_suggest_fix`). Table 11 summarizes the resulting specialization.

Table 11: Cross-Model Specialization by Issue Class

Issue Class	Opus	Sonnet+Codex
Architectural patterns (DRY, dispatch)	Strong	Moderate
N+1 query detection	Strong	Moderate
Design compliance verification	Strong	Weak
Race conditions / concurrency	Weak	Strong
Cross-tenant isolation gaps	Moderate	Strong
Prompt injection vectors	Moderate	Strong
Domain knowledge conflicts	Weak	Strong
Input validation (length, format)	Moderate	Strong

Similarly, Lina’s dual-model UI workflow (Claude + Gemini) identified RTL layout issues and color contrast failures that Claude alone did not catch in 2 of 4 design sessions—further evidence that cross-provider composition yields non-redundant coverage.

Finding: Dual independent reviews with differentiated prompts and models produce strongly complementary coverage (32.8% overlap, $n = 125$). Model diversity functions analogously to cognitive diversity in human review teams—Sonnet+Codex acts as the sharper security auditor, Opus as the stronger architect-reviewer. The combination catches 80% more issues than either model alone.

5.3 RQ3: Red Team Effectiveness

Table 12 shows results across 6 red team sessions.

Table 12: Red Team Attack Results ($n = 30$, 6 sessions)

Category	Att.	Block	Succ.	Fixed
Prompt Injection	8	6	2	2
Data Exfiltration	6	6	0	–
Privilege Escalation	5	2	3	3
Tenant Boundary	4	4	0	–
PII Leakage	4	3	1	1
API Security	3	1	2	2
Total	30	22	8	8

8 of 30 attacks (26.7%) succeeded against code that had passed all prior quality gates (architecture design, implementation, dual code review, and QA testing).

Table 13 details the severity distribution of successful attacks.

Table 13: Severity of Successful Red Team Attacks ($n = 8$)

Severity	Count	Examples
Critical	3	Unauthenticated admin API, self-approval via agent, balance manipulation
High	3	PII in error message, partial prompt extraction, IDOR
Medium	2	Off-topic prompt injection, rate limit absence

The most critical finding: **admin API endpoints with zero authentication**—full CRUD on sensitive records and approvals, accessible by anyone with a `curl` command. This vulnerability survived architecture design (the architect did not specify auth for admin routes), implementation (the developer followed the design without questioning the omission), dual code review (neither reviewer flagged missing authentication), and QA testing (QA tested functional correctness, not authorization). Only the adversarial red team agent—specifically probing for privilege escalation—identified the vulnerability.

Finding: Standard SDLC quality gates (design, review, QA) have systematic blind spots for authorization-layer vulnerabilities. Dedicated adversarial testing adds non-redundant value, with a 26.7% attack success rate against “finished” code.

5.4 RQ4: Utilization, Velocity, and Testing

5.4.1 Agent Utilization Patterns

Table 14: Agent Utilization Across 11 Sprints

Agent	Role	Sess.	Sprints	Category
Rania	Developer	~28	11/11	Daily driver
Khaled	Reviewer 1	~22	10/11	Daily driver
Layla	QA Engineer	~20	10/11	Daily driver
Faisal	Architect	~18	10/11	Daily driver
Nasser	Reviewer 2	~15	8/11	Regular
Tariq	Product Owner	~12	7/11	Sprint starts
Youssef	Prompt Tester	~10	7/11	Per-agent
Huda	Conv. Tester	~10	7/11	Per-agent
Zaid	Red Team	~8	5/11	Major features
Dana	UX Researcher	~5	4/11	Sprint starts
Lina	UI Designer	~5	4/11	Sprint starts
Sultan	DevOps	~4	3/11	Infrastructure
Maha	Doc Writer	~4	3/11	Post-feature
Majid	DBA	~3	3/11	Schema changes
Reema	Analytics	~2	2/11	Pre-launch
Aws	SRE	~2	1/11	Pre-launch

A clear utilization hierarchy emerges with three tiers:

- **Daily drivers** (4 agents, ~88 sessions)—Developer, Reviewer 1, QA, Architect. These form the minimum viable pipeline.
- **Regular contributors** (7 agents, ~65 sessions)—Reviewer 2, PO, Prompt Tester, Conv Tester, Red Team, UX, UI.

- **Insurance policies** (5 agents, ~15 sessions)—DevOps, Doc Writer, DBA, Analytics, SRE. Invoked rarely but provide infrastructure readiness and preparatory value.

5.4.2 Shipping Velocity

Table 15: Production Output Over 11 Sprints

Deliverable	Tools	Sprints	Passes	Reviews
Agent A (Primary workflows)	19	2	1	2
Agent B (Multi-step workflows)	14	4	1	2
Agent C (Domain pipeline)	28	3	1	2
Agent D (Operations)	17	1	1	2
Agent E (Executive analytics)	7	1	1	2
Total	85	11	5	10

Average velocity: **~8 tools per sprint**, with each tool passing dual review, QA testing, and (for major features) prompt testing, conversation testing, and red team assessment.

5.4.3 Conversation Testing Results

Table 16: Conversation Test Results (Final Run)

Metric	Value
Personas	6
Scenarios	13
Conversation turns	~78
Detectors per turn	11
Total checks	~858
Pass rate	94.2%
Failures	50

Table 17: Failure Analysis by Detector

Detector	Fails	Root Cause
Latency (>10s)	22	LLM response time variance
Language consistency	12	Agent switching AR→EN mid-conversation
Raw enum leakage	8	Raw enum values in user-facing text
Correct agent routing	5	Cross-agent intent misclassification
Other	3	Edge cases in natural date parsing

The `conv-sim` framework executed 858 automated checks across 78 conversation turns, achieving a 94.2% pass rate. The 50 failures cluster into two categories: *infrastructure-bound* and *agent-bound*. Latency failures (22/50, 44%) reflect LLM response time variance rather than application defects—these are non-deterministic and largely outside developer control. The remaining 28 failures expose genuine agent-level issues. Language consistency failures (12) occurred when agents defaulted to English mid-conversation despite an Arabic-speaking persona,

typically after tool execution returned English-formatted data. Raw enum leakage (8) revealed cases where internal status codes (e.g., `Status.PENDING`) surfaced in user-facing responses—a presentation-layer defect invisible to unit tests but immediately apparent to conversation-level detectors. Agent routing failures (5) involved misclassification of ambiguous intents that span multiple agent domains.

Two observations merit emphasis. First, conversation-level testing catches a class of defects that no other pipeline stage addresses: the interaction between LLM generation, tool execution, and user-facing formatting. Unit tests validate tool logic; code review validates patterns; but only end-to-end conversation simulation validates that a user asking a question in Arabic receives a coherent, correctly-formatted Arabic response with real data. Second, the detector architecture proves composable—adding a new detector (e.g., `mentions_calendar_conflict`) requires ~ 10 lines of Python and immediately applies to all 78 turns retroactively, creating a ratchet effect where conversation quality can only improve over time.

6 Discussion

Looking across all four research questions, a pattern emerges: the quality gains in this study did not come from using a better model. They came from how we structured the work. Restricting a reviewer’s tools changed its behavior more than any prompt rewrite did (RQ1). Pairing two different models for review caught 80% more issues than either one alone (RQ2). A dedicated red team agent broke into code that had passed architecture review, dual code review, and QA testing (RQ3). And utilization data showed that only 4–5 core agents are truly essential—the rest earn their keep in specific situations (RQ4). These are not properties of any individual model—they are properties of the *process*. What follows is our attempt to make sense of what we observed and what it might mean for others.

6.1 What Actually Changes When You Separate Roles

The most surprising result was not that role-separated reviewers produced better output—we expected that. The surprise was *how different* the output was. A reviewer with full tool access behaves like a developer who happens to be reviewing: it spots an issue, patches it inline, and moves on. The review is shallow because the agent can act on problems instead of explaining them. Remove the ability to edit, and the same model produces detailed, structured feedback—not because we asked it to, but because it has nowhere else to direct its effort.

We call this behavioral redirection, and it turned out to be the clearest evidence for why multi-agent separation matters. A monolithic assistant that alternates between writing and reviewing code is, in effect, reviewing its own work with the same biases it had when writing it. A separate reviewer agent—different session, different permissions, different prompt—breaks that cycle. The result is not a marginal improvement. It is a qualitatively different kind of feedback that a solo developer cannot produce for their own code, no matter how much time they spend.

This principle likely extends beyond software. Any domain where judgment should be independent from action—content moderation, financial analysis, medical triage—may benefit from agents that are constrained to reason rather than act.

6.2 Why Two Reviewers Are Not Redundant

Before running the dual-review experiment, we considered the possibility that two LLM reviewers would simply find the same issues with different wording. The 32.8% overlap rate proved otherwise. More interesting than the number itself was what each reviewer missed. Opus consistently caught architectural concerns—DRY violations, N+1 queries, dispatch patterns—but overlooked security boundaries. Sonnet+Codex, augmented with OpenAI’s tools, found race

conditions, cross-tenant isolation gaps, and prompt injection vectors that Opus never flagged. All five exclusive CRITICAL findings came from the cheaper model pairing.

This was not what we expected. The natural assumption is that the more expensive, more capable model would dominate. Instead, we found that the right question is not “which model is better?” but “better at what?” Opus and Sonnet+Codex have different blind spots, and those blind spots are complementary. This mirrors what we know about human review teams: a team with diverse perspectives catches more than a team of individually stronger but similarly-minded reviewers. The practical takeaway is simple—if you are going to invest in dual review, make the two reviewers as different as possible. Same model, same prompt, run twice, is a waste of budget.

6.3 The Case for Adversarial Testing by Default

Eight of thirty attacks succeeded against code that had passed architecture review, dual code review, and QA testing. Three of those were critical severity, including an admin API with zero authentication. That vulnerability is worth pausing on: the architect did not specify auth for admin routes, the developer followed the design faithfully, neither reviewer flagged the omission, and QA tested functionality rather than authorization. Four stages, four opportunities, and the issue sailed through all of them.

This is not a failure of any individual stage. Each stage did its job—design quality, code quality, test correctness. The problem is that none of them adopt an attacker’s mindset. Authorization-layer vulnerabilities live in the gaps between what each stage is optimized to find. The red team agent, with a 203-line attack playbook, costs almost nothing to run. For a compliance-sensitive product, that cost is negligible compared to shipping a vulnerability that an actual attacker would find in minutes.

6.4 What the Human Operator Actually Does

After 11 sprints, we can say with some confidence what the pipeline replaces and what it does not. It replaces the *execution* of code generation, review, testing, security auditing, and documentation. It does not replace the *judgment* that ties those activities together.

In practice, this means the operator spends most of their time on two things. The first is resolving ambiguity—deciding between competing architectural approaches, choosing which reviewer’s severity rating is more accurate, determining whether a failing conversation test reflects a real bug or a flaky detector. The second is writing and maintaining system prompts. The 1,800 lines of prompts across 16 agents encode domain knowledge—regulations, compliance rules, locale-specific logic—that the agents apply but cannot acquire on their own. This knowledge compounds: a rule written once applies to every future session, which is why domain-specific errors dropped dramatically over the course of the study.

An unexpected side effect was that the architect agent changed how we think about implementation. Before the pipeline, the workflow was “open a file, start coding, refactor as problems surface.” The architect forces a different discipline: articulate data models first, define API contracts, *then* implement. That habit persisted even in sessions where we skipped the architect, suggesting the pipeline may reshape how developers approach problems—what Bjork and Bjork [21] describe as “desirable difficulties” that improve learning.

The competency profile this implies is closer to a technical lead than a developer. The bottleneck is not “can you build it?” but “can you tell whether it was built correctly?” Deep expertise in any single area matters less than the ability to evaluate across architecture, security, UX, and domain correctness.

6.5 What This Means for Small Teams and Solo Founders

Three observations stand out for early-stage organizations. First, the minimum viable engineering team may be smaller than commonly assumed. One operator with deep domain knowledge ran a full pipeline—independent review, adversarial testing, structured QA—for 11 sprints. This does not mean teams are unnecessary; it means the need for a team can be deferred until the product reaches a stage where non-engineering functions (sales, customer support, fundraising) become the bottleneck.

Second, the pipeline amplifies expertise but does not create it. An operator who understands the target domain deeply can write 100+ lines of compliance rules and edge cases per agent. An operator without that understanding will write generic prompts and get generic output. The agents are only as good as the knowledge encoded in their prompts.

Third, cost shifts from headcount to compute. The pipeline loads roughly 45K tokens of system prompts per invocation, with quality gates running concurrently. At current API pricing, this is a fraction of even a small engineering team’s cost. But as Kapoor et al. [26] rightly argue, the meaningful comparison is not cost per token—it is cost per shipped feature at a given quality bar.

6.6 Where This Breaks Down

We do not want to overstate what this study demonstrates. Four failure modes recurred throughout the 11 sprints. Context loss in manual handoff happened roughly once per sprint—the operator forgot to pass a constraint from one agent to the next, and downstream stages inherited silent assumptions. Prompt drift set in during long sessions (beyond 30 turns), with agents gradually abandoning structured output formats. File-path references in prompts went stale after refactoring. And some model-specific blind spots proved persistent—Claude consistently missed timezone handling, which is part of why we adopted cross-model composition in the first place.

More fundamentally, this is a single project built by a single developer, without a controlled baseline. We cannot say with certainty that the multi-agent pipeline produced better outcomes than a skilled developer with a monolithic assistant would have. The utilization data is self-reported, not instrumented. Survivorship bias is real—we are reporting on an approach that worked, and failed experiments may look very different. What we can say is that the pipeline was feasible, that it produced measurable quality signals (review overlap, red team results, conversation test pass rates), and that several of those signals surprised us.

6.7 Seven Principles That Held Up

Over the course of the study, seven design principles proved consistently useful:

1. **Remove capabilities rather than adding instructions.** The behavioral difference between a reviewer that *cannot* edit and one told not to edit is not subtle—it is measurable across every metric we tracked.
2. **Make reviewers deliberately different.** Same model, same prompt, run twice, produces high overlap and low marginal value. Differentiate by model, prompt focus, and tool access.
3. **Invest in long system prompts.** 100+ lines of domain knowledge per agent feels excessive at first, but the payoff compounds over every future session.
4. **Standardize output formats early.** Tables, severity tags, verdict formats—downstream agents depend on upstream consistency. Fixing this later is painful.

5. **Include adversarial testing as a default phase.** Not as an occasional audit, but as a standard step for every major feature.
6. **Run independent stages in parallel.** Phases 3 and 4 of our pipeline execute concurrently— $3\times$ the token cost, $1\times$ the wall-clock time, $3\times$ the coverage.
7. **Start small.** Four or five core agents (developer, architect, reviewer, QA) form the minimum viable pipeline. Add specialists only when you feel the pain of not having them.

Four anti-patterns caused the most wasted effort: monolithic agents that try to do everything (role contamination is real), prompt-only role enforcement (it drifts within a single session), symmetric dual review (expensive redundancy), and provisioning agents before validating their need (idle agents accumulate stale prompts).

6.8 What We Would Study Next

The most pressing engineering problem is the human orchestrator. Manual handoff between phases is the pipeline’s weakest link and the source of its most common failure mode. Agent-to-agent communication via shared artifacts—design documents, review verdicts, test reports—would address this directly. Persistent memory across sessions, so agents retain knowledge of prior decisions and resolved bugs, would eliminate the frustrating pattern of agents rediscovering domain constraints that were already handled in earlier sprints.

On the optimization side, cost-tiered model routing deserves rigorous study. Our data hints that Haiku might handle style checks, Sonnet+Codex security review, and Opus architectural analysis, but we did not test this systematically. The optimal number of reviewers is also an open question—two gave us 32.8% overlap, but would a third reviewer push that below 20%, or would diminishing returns take over?

The most important question we cannot answer from this study is whether the multi-agent approach genuinely outperforms a skilled developer with a monolithic assistant. A controlled comparison—same developer, same requirements, same timeline, both approaches—would settle this. Beyond that, we are curious whether the architecture transfers to other regulated domains: healthcare with HIPAA constraints, finance with SOX controls, legal with contract review requirements. The role decomposition and tool-scoping principles feel general, but we have only tested them in one domain, and generality is an empirical question.

Acknowledgments

This paper grew out of a practical need: as an independent researcher and entrepreneur building an AI-native business, I wanted to document what I was learning in real time. The field of multi-agent LLM systems is moving fast, and much of the practitioner knowledge—what works, what fails, what surprises you—lives in blog posts and X posts rather than in empirical studies. My background is academic, but my current work is in industry, and this paper is an attempt to bridge the two: to bring the rigor of empirical research to findings that emerged from building a production system under real constraints.

I am grateful to Dr. Steve Maddock for his kind review and suggestions, which significantly improved the structure and presentation of this paper.

References

- [1] M. Beller, A. Bacchelli, A. Zaidman, and E. Juergens, “Modern code reviews in open-source projects: Which problems do they fix?” in *Proc. MSR*, 2014.

- [2] M. Chen et al., “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
- [3] R. Li et al., “StarCoder: May the source be with you!” *arXiv:2305.06161*, 2023.
- [4] Z. Li et al., “Automating code review activities by large-scale pre-training,” in *Proc. FSE*, 2022.
- [5] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, “Using pre-trained models to boost code review automation,” in *Proc. ICSE*, 2022.
- [6] C. Lemieux et al., “CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models,” in *Proc. ICSE*, 2023.
- [7] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, “Examining zero-shot vulnerability repair with large language models,” in *Proc. IEEE S&P*, 2023.
- [8] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv:2308.08155*, 2023.
- [9] S. Hong et al., “MetaGPT: Meta programming for multi-agent collaborative framework,” *arXiv:2308.00352*, 2023.
- [10] C. Qian et al., “ChatDev: Communicative agents for software development,” *arXiv:2307.07924*, 2023.
- [11] S. Lu et al., “LLM-based code review: A systematic literature review,” *arXiv:2307.12929*, 2023.
- [12] F. Perez and I. Ribeiro, “Ignore this title and HackAPrompt: Evaluating prompt injection in large language models,” *arXiv:2211.09527*, 2022.
- [13] S. Schulhoff et al., “Ignore this title and HackAPrompt: Exposing systemic weaknesses of LLMs through a global prompt hacking competition,” in *Proc. EMNLP*, 2023.
- [14] K. Greshake et al., “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. AISec*, 2023.
- [15] OWASP, “OWASP Top 10 for LLM Applications,” 2025.
- [16] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [17] D. Huang et al., “AgentCoder: Multi-agent-based code generation with iterative testing and optimisation,” *arXiv:2312.13010*, 2024.
- [18] G. Deng et al., “MasterKey: Automated jailbreaking of large language model chatbots,” in *Proc. NDSS*, 2024.
- [19] S. Gururangan et al., “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proc. ACL*, 2020.
- [20] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [21] E. L. Bjork and R. A. Bjork, “Making things hard on yourself, but in a good way: Creating desirable difficulties to enhance learning,” in *Psychology and the Real World*, 2011.

- [22] J. Yang et al., “SWE-agent: Agent-computer interfaces enable automated software engineering,” *arXiv:2405.15793*, 2024.
- [23] X. Wang et al., “OpenHands: An open platform for AI software developers as generalist agents,” *arXiv:2407.16741*, 2024.
- [24] M. N. Islam et al., “MapCoder: Multi-agent code generation for competitive programming,” in *Proc. ACL*, 2024.
- [25] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in *Proc. NeurIPS*, 2023.
- [26] S. Kapoor et al., “AI agents that matter,” *arXiv:2407.01502*, 2024.
- [27] J. Liu et al., “Large language model-based agents for software engineering: A survey,” *arXiv:2409.02977*, 2024.